

Project 1

Exploring SocketCAN – Software-Only CAN/CAN FD Communication on Linux

Submitted by: Prajakta Laxmikant Dhole. BE208

Department of Electronics and Telecommunication Engineering

Marathawada Mitra Mandal's College of Engineering

21 August 2026

1. Introduction

Controller Area Network (CAN) is widely used in automotive and embedded systems for communication between Electronic Control Units (ECUs). Traditionally, CAN applications require physical CAN hardware for development and testing. Linux provides SocketCAN, a networking framework that allows CAN communication to be handled using standard Linux networking concepts.

This project investigates SocketCAN and demonstrates CAN and CAN FD communication without external CAN hardware. A Virtual CAN (`vcan`) interface is used to create a software-only CAN network. Multiple software ECUs are developed in C, including a Vehicle ECU, Dashboard ECU and Logger ECU.

The Vehicle ECU generates vehicle parameters such as vehicle speed, engine RPM and coolant temperature. The Dashboard ECU receives and decodes these messages, while the Logger ECU records CAN traffic into a CSV file. Additional experiments demonstrate message filtering, unknown message detection, transmission-rate analysis, node failure detection, basic diagnostics and CAN FD communication.

2. Objectives

- To understand the Linux SocketCAN framework.
- To create a software-only CAN network using a Virtual CAN interface.
- To develop multiple CAN communication nodes using C.
- To implement CAN message transmission, reception, filtering, and logging.
- To study node failure and basic diagnostic mechanisms.
- To explore CAN FD communication and larger payloads.

3. SocketCAN

SocketCAN is the CAN networking subsystem provided by the Linux kernel. It provides a socket-based interface for CAN communication, allowing applications to communicate with CAN interfaces using standard Linux socket APIs.

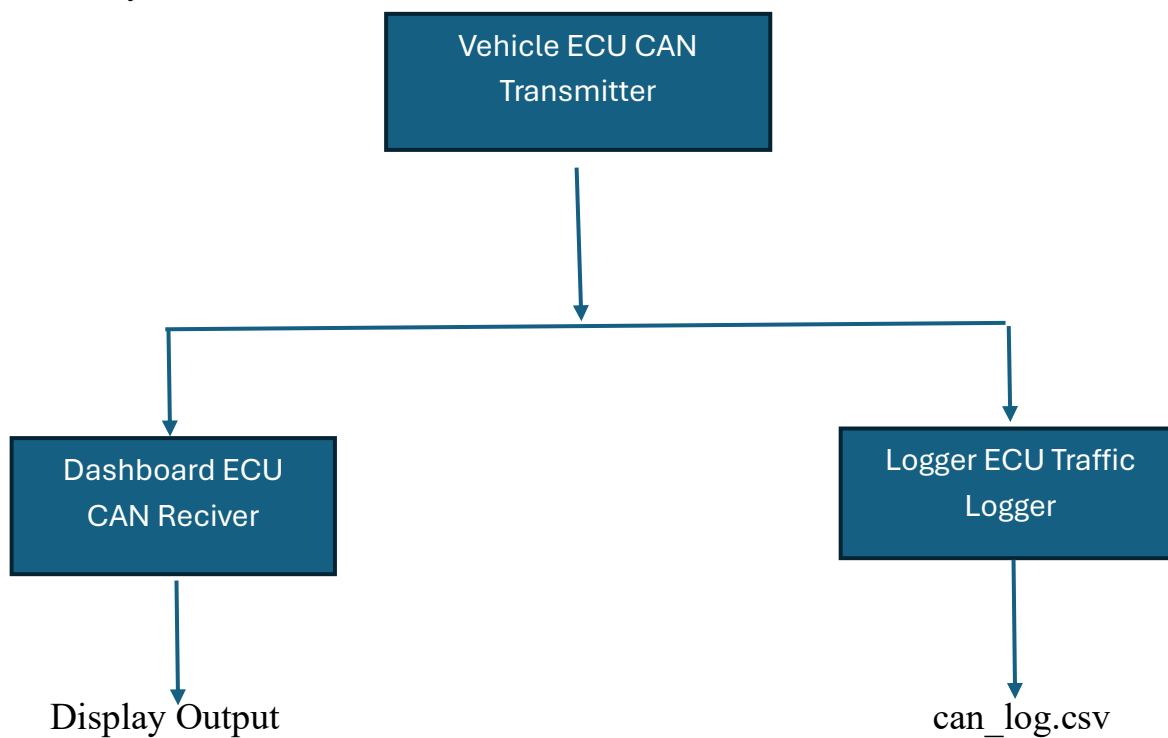
CAN is represented as a network interface in Linux. This allows applications to use familiar networking mechanisms such as sockets, interfaces and filtering. Applications do not need to depend on a specific manufacturer's proprietary API.

One major advantage of SocketCAN is that multiple applications can access the same CAN network simultaneously. For example, the Dashboard ECU can decode messages while the Logger ECU records the same traffic and `candump` monitors it.

For this project, the `vcan` driver was used. Virtual CAN allows CAN applications to be developed and tested without physical CAN hardware.

Feature	SocketCAN
Operating System	Linux
Interface	CAN network interface
API	Linux socket API
Physical Hardware	Not required
Traffic monitoring	candump
CAN filtering	CAN_RAW_FILTER
CAN FD	Demonstrated

3.1 System Architecture :



The Vehicle ECU acts as the transmitter and periodically sends three CAN messages through vcan0. The Dashboard ECU receives and decodes the messages and displays the corresponding vehicle parameters. The Logger ECU observes CAN traffic and records the timestamp, identifier, payload length and payload data in a CSV file.

4. CAN Message Definition

CAN ID	Signal	DLC	Unit
0x100	Vehicle Speed	2 bytes	km/h
0x101	Engine RPM	2 bytes	rpm
0x102	Coolant Temperature	1 byte	°C

4.1 Vehicle Speed

CAN ID **0x100** is used for vehicle speed. The speed value is encoded using two bytes.

$$\text{Speed} = (\text{Data}[0] \ll 8) \mid \text{Data}[1]$$

4.2 Engine RPM

CAN ID **0x101** is used for engine RPM. The RPM value is encoded using two data bytes.

$$\text{RPM} = (\text{Data}[0] \ll 8) \mid \text{Data}[1]$$

4.3 Coolant Temperature

CAN ID **0x102** is used for coolant temperature. The first data byte contains the temperature value.

$$\text{Temperature} = \text{Data}[0]$$

5. Implementation

5.1 Vehicle ECU

The Vehicle ECU creates a CAN RAW socket and connects it to the `vcan0` interface. It generates vehicle speed, engine RPM and coolant temperature values and periodically transmits them using CAN frames. The values are varied over time to simulate changing vehicle conditions.

5.2 Dashboard ECU

The Dashboard ECU receives CAN frames from `vcan0`, identifies the CAN ID and decodes the corresponding payload. It displays the decoded vehicle parameters. SocketCAN's `CAN_RAW_FILTER` mechanism is used to implement message filtering. The Dashboard was tested for receiving all messages and for individual Speed and RPM filters.

5.3 Logger ECU

The Logger ECU monitors CAN traffic and stores received frames in a CSV file. Each entry contains the timestamp, CAN identifier, payload length and payload data.

```
prajakta-dhole@prajakta-dhole-HP-205-G1-A10-Business-PC:~/socketcan_project$ ./dashboard_ecu all
Dashboard filter: ALL messages
Dashboard ECU started...

Speed received: 65 km/h
RPM received: 4300 rpm
Temperature received: 75 C
Speed received: 70 km/h
RPM received: 4500 rpm
Temperature received: 76 C
Speed received: 75 km/h
RPM received: 4700 rpm
Temperature received: 77 C
Speed received: 80 km/h
RPM received: 4900 rpm
Temperature received: 78 C
Speed received: 85 km/h
RPM received: 1500 rpm
Temperature received: 79 C
Speed received: 90 km/h
RPM received: 1700 rpm
Temperature received: 80 C
Speed received: 95 km/h
RPM received: 1900 rpm
Temperature received: 81 C
Speed received: 100 km/h
RPM received: 2100 rpm
```

Figure 1: Dashboard ECU displaying decoded CAN data

```
prajakta-dhole@prajakta-dhole-HP-205-G1-A10-Business-PC:~/socketcan_project$ cat can_log.csv
Timestamp,CAN_ID,DLC,Data
2026-08-16 23:50:05,0x100,2,00 2D
2026-08-16 23:50:05,0x101,2,13 24
2026-08-16 23:50:05,0x102,1,69
2026-08-16 23:50:06,0x100,2,00 32
2026-08-16 23:50:06,0x101,2,05 DC
2026-08-16 23:50:06,0x102,1,6A
2026-08-16 23:50:07,0x100,2,00 37
2026-08-16 23:50:07,0x101,2,06 A4
2026-08-16 23:50:07,0x102,1,6B
2026-08-16 23:50:08,0x100,2,00 3C
2026-08-16 23:50:08,0x101,2,07 6C
2026-08-16 23:50:08,0x102,1,6C
2026-08-16 23:50:09,0x100,2,00 41
2026-08-16 23:50:09,0x101,2,08 34
2026-08-16 23:50:09,0x102,1,6D
2026-08-16 23:50:10,0x100,2,00 46
2026-08-16 23:50:10,0x101,2,08 FC
2026-08-16 23:50:10,0x102,1,6E
2026-08-16 23:50:11,0x100,2,00 4B
2026-08-16 23:50:11,0x101,2,09 C4
2026-08-16 23:50:11,0x102,1,6F
2026-08-16 23:50:12,0x100,2,00 50
2026-08-16 23:50:12,0x101,2,0A 8C
2026-08-16 23:50:12,0x102,1,70
2026-08-16 23:50:13,0x100,2,00 55
2026-08-16 23:50:13,0x101,2,0B 54
2026-08-16 23:50:13,0x102,1,71
```

Figure 2: CAN traffic recorded in CSV format

6. Learning Challenges and Results

6.1 Traffic Observation

Multiple applications were connected to the same `vcan0` interface. The Dashboard ECU, Logger ECU, and `candump` successfully observed the same CAN traffic simultaneously. Communication continued normally while the Logger ECU was active.

Result: Multiple applications can observe the same CAN traffic simultaneously through SocketCAN.

6.2 Message Filtering

CAN RAW filters were implemented in the Dashboard ECU using the `CAN_RAW_FILTER` mechanism. The Dashboard was tested with Speed and RPM filters. When a specific filter was enabled, only the corresponding CAN messages were processed by the Dashboard ECU.

Result: SocketCAN supports CAN identifier-based message filtering at the socket level.

6.3 Unknown Message Detection

An additional CAN message with an undefined CAN identifier was transmitted on the `vcan0` interface. The Logger ECU successfully recorded the unknown message. When the Dashboard ECU was configured to receive all messages, it identified the message as an unknown CAN ID.

Result: Unknown CAN messages can be detected and logged without affecting the defined CAN messages.

6.4 Transmission Rate Study

The transmission interval of the Vehicle ECU was modified to study the effect of different transmission rates. A 100 ms transmission experiment was performed. A shorter transmission interval generated more CAN frames within the same period, resulting in increased traffic and faster growth of the log file.

Result: The transmission rate directly affects CAN traffic volume, receiver activity, and log-file growth.

6.5 Node Failure Study

The Vehicle ECU was terminated while the Dashboard ECU and Logger ECU remained active. After the Vehicle ECU stopped transmitting, the Dashboard ECU stopped receiving new vehicle messages, and the Logger ECU stopped recording new messages from the Vehicle ECU.

Result: Loss of CAN communication can be detected by monitoring the time since the last expected message. An application-level timeout mechanism can be used for node failure detection.

6.6 Basic Diagnostics

A basic diagnostic mechanism was implemented to detect the loss of Vehicle ECU communication. The Dashboard ECU monitors the reception of Speed messages. If a Speed message is not received within the specified timeout period, the system generates an offline warning.

Diagnostic Output:

WARNING: Vehicle ECU Offline

Result: The diagnostic mechanism successfully detects the absence of expected CAN messages and provides an application-level warning indicating that the Vehicle ECU may be offline.

7. CAN FD Exploration

CAN FD extends Classical CAN by supporting larger payloads and flexible data rates. Classical CAN supports a maximum payload of 8 bytes, whereas CAN FD supports payloads of up to 64 bytes.

In this project, CAN FD communication was demonstrated using the virtual CAN environment. A CAN FD sender and receiver were developed and tested. A 16-byte CAN FD frame was successfully transmitted and received.

```
prajakta-dhole@prajakta-dhole-HP-205-G1-A10-Business-PC:~/socketcan_project$ candump vcanfd0
vcanfd0 123 [1] 11
read: Network is down
prajakta-dhole@prajakta-dhole-HP-205-G1-A10-Business-PC:~/socketcan_project$ candump vcanfd0
vcanfd0 123 [04] 11 22 23 33
vcanfd0 123 [16] 11 22 23 33 44 45 55 66 67 77 88 89 99 00 00 00
vcanfd0 123 [16] 11 22 23 33 44 45 55 66 67 77 88 89 99 AA BB 00
vcanfd0 123 [20] 11 22 23 33 44 45 55 66 67 77 88 89 99 AA BB CC DD 00 00 00
vcanfd0 123 [20] 11 22 23 33 44 45 55 66 67 77 88 89 99 AA BB CC DD EE 00 00
vcanfd0 123 [20] 11 22 23 33 44 45 55 66 67 77 88 89 99 AA BB CC DD EE FF 00
vcanfd0 123 [20] 11 22 23 33 44 45 55 66 67 77 88 89 99 AA BB CC DD EE FF 00
```

Figure 3: Successful reception of a 16-byte CAN FD frame

Parameter	Classical CAN	CAN FD
Maximum payload	8 bytes	64 bytes
Flexible data rate	No	Yes
Larger payload	No	Yes

8. Conclusion

This project successfully demonstrated software-only CAN communication using Linux SocketCAN. A Virtual CAN interface was used to create a CAN network without requiring physical CAN hardware. Multiple software ECUs were developed to demonstrate vehicle data transmission, reception, decoding and logging.

The project also demonstrated CAN message filtering, unknown message detection, transmission-rate analysis, node failure detection and basic diagnostics. CAN FD communication was additionally explored, and a 16-byte CAN FD frame was successfully transmitted and received.

The project provided practical understanding of Linux SocketCAN, CAN RAW sockets, Virtual CAN interfaces, CAN frame handling, message filtering, traffic logging and CAN FD communication. The software-only approach demonstrates how CAN applications can be developed and tested before physical automotive hardware is available.